TASKUP: GERENCIADOR DE TAREFAS

Sabrina Rayssa Teixeira
Prof. Ricardo Bicalho

RESUMO

Este artigo apresenta o TaskUp, um gerenciador de tarefas pessoais formado por uma API RESTful e por uma interface web que a consome. O objetivo do trabalho é praticar desenvolvimento backend, modelagem de banco de dados, criação de APIs REST e autenticação de usuários em um sistema pequeno. A API foi feita em Node.js com Express e usa o SQLite como banco de dados. O cadastro solicita nome, telefone, e-mail e senha; a senha é armazenada como hash com salt, e o login é feito por e-mail. O sistema também permite recuperar a senha confirmando o e-mail e o telefone cadastrados. Com a interface é possível organizar tarefas em categorias, criar checklists de subtarefas, visualizar detalhes, buscar e filtrar tarefas, concluir e excluir tarefas. A exclusão funciona como uma lixeira, permitindo restaurar a tarefa antes de apagá-la definitivamente. O sistema foi publicado no Render Cloud e está disponível em https://taskup-rffb.onrender.com/.

Palavras-chave: API REST; Node.js; Express; SQLite; autenticação; gerenciamento de tarefas.

ABSTRACT

This paper presents TaskUp, a personal task manager composed of a RESTful API and a web interface that consumes it. The goal of the work is to practice backend development, database modeling, REST API design and user authentication in a small system. The API was built with Node.js and Express and uses SQLite as its database. Registration requires name, phone number, e-mail and password; the password is stored as a salted hash, and login is performed with the e-mail. The system also allows password recovery by confirming the registered e-mail and phone number. The interface lets users organize tasks into categories, create subtask checklists, view details, search and filter tasks, complete and delete tasks. Deletion works as a trash bin, so a task can be restored before being permanently removed. The system is deployed on Render Cloud at https://taskup-rffb.onrender.com/.

Keywords: REST API; Node.js; Express; SQLite; authentication; task management.

1. INTRODUÇÃO

Um gerenciador de tarefas é uma aplicação fácil de entender e boa para praticar os conceitos da disciplina. O cliente, que é a interface, fica separado do servidor, que guarda os dados e aplica as regras, e os dois conversam por meio de uma API. Assim, a mesma API pode atender uma interface web ou um aplicativo móvel.

O TaskUp foi desenvolvido na unidade curricular Sistemas Distribuídos e Mobile. O objetivo é criar uma aplicação para organizar tarefas e, com ela, aplicar conceitos de backend, banco de dados, APIs RESTful e autenticação. Como metas, o trabalho definiu implementar as operações básicas sobre as tarefas, cadastrar usuários com senha protegida, permitir login por e-mail e recuperação de senha, integrar o backend a uma interface web e acrescentar recursos de organização, como categorias, subtarefas, prazos, prioridades, filtros e uma lixeira para exclusões reversíveis.

2. FUNDAMENTAÇÃO TEÓRICA E TECNOLOGIAS

2.1. API RESTful

REST é um estilo de arquitetura apresentado por Fielding (2000). Nele, cada coisa que a aplicação manipula é um recurso com um endereço (URL), e as operações usam os métodos do HTTP: GET para consultar, POST para criar, PUT para alterar e DELETE para excluir. A resposta traz um código de status, como 200 (sucesso), 201 (criado), 204 (sem conteúdo), 400 (requisição inválida), 401 (não autenticado), 404 (não

encontrado) e 409 (conflito). No TaskUp, os recursos principais são usuários, categorias, tarefas e subtarefas, acessados por rotas como `/usuarios`, `/categoria` e `/tarefas`.

2.2. Node.js, Express e SQLite

Foram usados o Node.js, que executa JavaScript no servidor (NODE.JS, [s.d.]), e o Express, um framework que simplifica a criação de rotas (EXPRESS, [s.d.]). O banco de dados é o SQLite, que guarda tudo em um único arquivo e não exige instalar um servidor de banco (SQLITE, [s.d.]). Ele é acessado pelo módulo `node:sqlite`, disponível a partir do Node.js 22.5. Por isso, as dependências do projeto continuam pequenas: Express e CORS, este último para liberar o acesso à API a partir de outra origem, como o frontend. O projeto exige Node.js 22.5 ou superior.

2.3. Autenticação e proteção de senhas

A senha nunca é gravada em texto puro. O backend utiliza a função `scrypt` do módulo nativo `node:crypto`, com um salt aleatório para cada usuário. O banco guarda apenas o resultado no formato `salt:hash`. No login, a senha digitada é processada com o mesmo salt e comparada com o hash usando `timingSafeEqual`, que evita diferenças de tempo que poderiam revelar informação.

O login é feito com e-mail e senha. O e-mail é normalizado para minúsculas e precisa ser único no banco. Para não revelar quais contas existem, o login devolve a mesma mensagem para e-mail inexistente e senha incorreta. O banco mantém a chave composta original `(nome, telefone)` porque as tarefas e categorias já utilizam esses campos; a coluna `email` foi adicionada por migração, com um índice único, sem apagar os usuários existentes.

A recuperação de senha adotada nesta versão é direta: a pessoa informa o e-mail, o telefone cadastrado e a nova senha. O telefone é comparado apenas pelos dígitos, então `(31) 97503-3481` e `31975033481` são equivalentes. Se os dois dados conferirem, a nova senha é gravada com o mesmo processo de hash. Esse fluxo é simples e não depende de envio de e-mail, mas é menos seguro que um link ou código temporário, pois quem conhecer os dois dados poderá redefinir a senha. A evolução natural é adicionar um token de uso único e limite de tentativas.

3. MODELAGEM E IMPLEMENTAÇÃO

3.1. Funcionalidades

As funcionalidades do TaskUp estão na Tabela 1 e são atendidas pelos endpoints da Tabela 2. O sistema foi pensado para ser simples de usar: cada tarefa pertence a um usuário, pode ter uma categoria, um prazo, uma prioridade e um checklist de subtarefas. A categoria é opcional; quando uma categoria é excluída, as tarefas continuam existindo, mas ficam sem categoria. A busca ignora diferenças de maiúsculas, minúsculas e acentos, e os filtros de status, categoria e busca podem ser combinados. No celular, a navegação principal passa para uma barra inferior, facilitando o uso em telas pequenas.

Além disso, ao clicar em um cartão na lista, a interface expande a descrição, a prioridade, o prazo e as subtarefas da tarefa. Nesse painel, o usuário pode marcar ou desmarcar uma subtarefa e acompanhar o progresso do checklist sem abrir outra janela. O botão “Subtarefas” continua disponível para adicionar ou remover itens. No formulário de tarefa, o botão “+ Nova” permite criar uma categoria sem sair do modal, e os ícones da navegação lateral foram padronizados em SVG.

Tabela 1 – Funcionalidades do TaskUp.

Código	Funcionalidade	Descrição
RF01	Cadastrar e autenticar usuário	Cria a conta com nome, telefone, e-mail e senha. A senha é gravada como hash com salt; o login é feito com e-mail e senha.
RF02	Recuperar senha	Confirma o e-mail e o telefone cadastrados e grava uma nova senha, sem depender de envio de e-mail.
RF03	Cadastrar tarefa	Cria uma tarefa com título obrigatório e descrição, prazo, prioridade e categoria opcionais. A tarefa nasce com status pendente.
RF04	Listar tarefas	Retorna as tarefas ativas do usuário, das mais recentes para as mais antigas.
RF05	Visualizar detalhes	Retorna os dados de uma tarefa específica.
RF06	Editar tarefa	Altera título, descrição, prazo, prioridade, categoria ou status.
RF07	Marcar como concluída	Muda o status para concluída e permite reabrir a tarefa.
RF08	Excluir tarefa	Move a tarefa para a lixeira, sem apagá-la imediatamente.
RF09	Restaurar ou excluir definitivamente	Na lixeira, permite restaurar a tarefa ou apagá-la de vez. A exclusão definitiva apaga as subtarefas em cascata.
RF10	Gerenciar categorias	Permite cadastrar, listar, renomear e excluir categorias, associar tarefas a uma categoria e filtrar a listagem. Também permite criar uma categoria rapidamente pelo formulário de tarefa.
RF11	Gerenciar subtarefas	Permite adicionar, concluir e remover subtarefas de uma tarefa, formando um checklist com progresso.
RF12	Filtrar e buscar tarefas	Permite filtrar por status e por categoria e buscar pelo título sem diferenciar maiúsculas ou acentos. Os filtros podem ser combinados.
RF13	Visualizar detalhes na lista	Ao clicar no cartão, exibe descrição, prioridade, prazo e subtarefas, permitindo concluir itens do checklist sem abrir outra janela.

Fonte: Elaborado pelos autores (2026).

3.2. Modelo de dados

O banco possui quatro tabelas relacionadas. `usuarios` armazena nome, telefone, e-mail, hash da senha e data de criação, com chave primária composta por nome e telefone e índice único para o e-mail. `categorias` pertence a um usuário e armazena o nome da categoria. `tarefas` armazena título, descrição opcional, prazo opcional, prioridade, status e data de criação, além de referenciar o usuário por nome e telefone e, opcionalmente, uma categoria. `subtarefas` armazena o título, o indicador de concluída e a data de criação de cada item do checklist, referenciando a tarefa.

A coluna `excluida_em` implementa a lixeira: quando está vazia, a tarefa aparece na listagem normal; quando recebe uma data, a tarefa passa a aparecer apenas na lixeira. Quando uma categoria é excluída, as tarefas ligadas a ela ficam sem categoria. Quando uma tarefa é apagada definitivamente, suas subtarefas são apagadas em cascata; enquanto a tarefa está na lixeira, as subtarefas são preservadas para a restauração. A migração do e-mail foi feita com `ALTER TABLE` e `CREATE UNIQUE INDEX`, preservando os registros antigos.

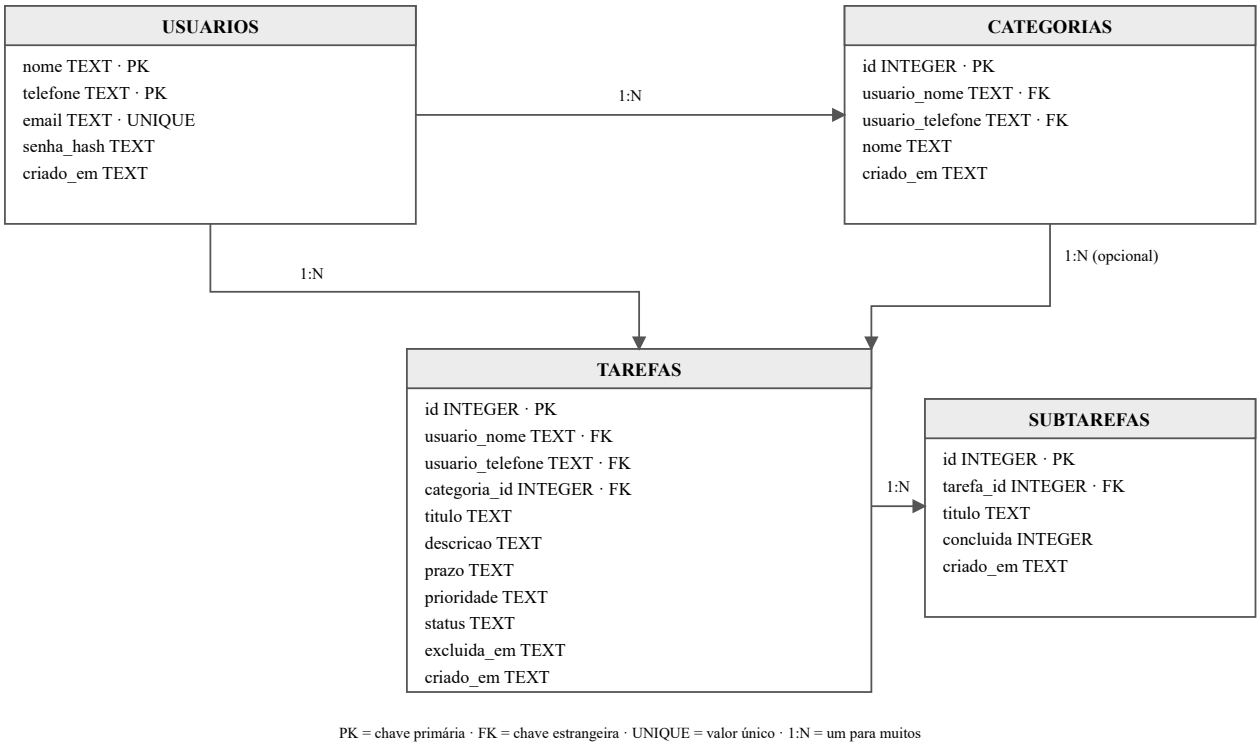


Figura 1 – Diagrama entidade-relacionamento do TaskUp.

Fonte: Elaborado pelos autores (2026).

As relações são: um usuário possui várias tarefas (1:N) e várias categorias (1:N); uma categoria pode estar em várias tarefas (1:N), e a tarefa pode não ter categoria; uma tarefa possui várias subtarefas (1:N), apagadas em cascata quando a tarefa é excluída definitivamente.

3.3. API do backend

O arquivo `db.js` cria as tabelas e aplica pequenas migrações; o `server.js` apenas registra os routers. Cada domínio tem sua própria pasta em `src`, com as consultas SQL (`sql.js`) separadas das rotas (`routes.js`). A Tabela 2 resume os endpoints.

Tabela 2 – Endpoints principais da API.

Método	Rota	Função	Respostas
GET	`/usuarios`	Lista usuários	200
POST	`/usuarios`	Cadastra usuário com nome, telefone, e-mail e senha	201, 400, 409
POST	`/usuarios/login`	Confere e-mail e senha	200, 400, 401
POST	`/usuarios/redefinir-senha-simples`	Confere e-mail + telefone e grava a nova senha	200, 400, 401
GET	`/categoria`	Lista categorias	200, 400
POST	`/categoria`	Cadastra categoria	201, 400, 404, 409
PUT	`/categoria/:id`	Edita categoria	200, 400, 404, 409
DELETE	`/categoria/:id`	Exclui categoria	204, 404
GET	`/tarefas`	Lista tarefas ativas	200, 400
GET	`/tarefas/lixeira`	Lista tarefas da lixeira	200, 400
GET	`/tarefas/:id`	Detalha tarefa	200, 404

Método	Rota	Função	Respostas
POST	`/tarefas`	Cadastra tarefa	201, 400, 404
PUT	`/tarefas/:id`	Edita tarefa e status	200, 400, 404
PUT	`/tarefas/:id/restaurar`	Restaura tarefa da lixeira	200, 400, 404
DELETE	`/tarefas/:id`	Move a tarefa para a lixeira	204, 404
DELETE	`/tarefas/:id/definitivo`	Apaga a tarefa do banco	204, 400, 404
GET	`/tarefas/:id/subtarefas`	Lista as subtarefas da tarefa	200, 404
POST	`/tarefas/:id/subtarefas`	Cadastra uma subtarefa	201, 400, 404
PUT	`/tarefas/:id/subtarefas/:subtarefaId`	Edita ou conclui uma subtarefa	200, 400, 404
DELETE	`/tarefas/:id/subtarefas/:subtarefaId`	Exclui uma subtarefa	204, 400, 404

Fonte: Elaborado pelos autores (2026).

O servidor valida o título obrigatório, o status (pendente ou concluída), a prioridade (baixa, média ou alta) e o formato do prazo (AAAA-MM-DD). A categoria é opcional, mas quando informada precisa pertencer ao mesmo usuário da tarefa. O título da subtarefa é obrigatório e a subtarefa precisa pertencer à tarefa informada na rota. Tarefas que estão na lixeira não aceitam alterações nas subtarefas: é preciso restaurá-las antes. A rota de exclusão definitiva só aceita tarefas que já estão na lixeira. No cadastro de usuário, nome, telefone, e-mail e senha são obrigatórios; o e-mail precisa ter formato válido e não pode repetir. A recuperação de senha exige que e-mail e telefone pertençam à mesma conta.

3.4. Interface web

A interface foi feita com HTML, CSS e JavaScript puro, sem framework. O arquivo `config.js` define o endereço da API; `script.js` concentra o estado da aplicação, as chamadas à API e a renderização das telas; `index.html` organiza as views e os modais; `style.css` cuida do layout e da responsividade. Há três views principais: Minhas tarefas, Concluídas e Lixeira. Os modais permitem criar e editar tarefas, gerenciar categorias e adicionar ou remover subtarefas.

O cadastro pede nome, telefone, e-mail e senha; no login, basta informar e-mail e senha. O usuário fica salvo no `localStorage` para as próximas visitas. A tela de login também oferece “Esqueci minha senha”, que abre um formulário para confirmar e-mail e telefone e definir uma nova senha. No formulário de tarefa, o botão “+ Nova” cria uma categoria sem fechar o modal. A navegação lateral usa ícones SVG padronizados, que herdam a cor do item ativo, e no celular a navegação passa para uma barra inferior. Ao trocar de view, a página volta ao topo para mostrar o resumo geral. Na publicação, o próprio backend entrega os arquivos do frontend, de modo que a mesma URL atende à interface e à API.

4. RESULTADOS PARCIAIS

4.1. Teste do backend e da interface

O backend e a interface foram executados localmente e testados com requisições HTTP e pelo navegador. O cadastro de usuário retornou 201 e um cadastro com e-mail repetido retornou 409. O login com e-mail e senha corretos retornou 200; senha errada retornou 401. A recuperação de senha com e-mail e telefone corretos retornou 200; telefone errado retornou 401 e senha com menos de seis caracteres retornou 400. Depois da redefinição, a senha antiga deixou de funcionar e a nova passou a autenticar. Também foi verificado que o telefone pode ser informado com ou sem formatação.

Na interface, foram testados login, recuperação de senha, gerenciamento de categorias, associação e filtro por categoria, checklist de subtarefas com progresso, busca por título sem diferenciar maiúsculas ou acentos, filtro por status, combinação de filtros, busca na lixeira, criação de categoria pelo formulário de tarefa, navegação com os novos ícones, navegação pelo menu inferior em tela pequena, criação, edição, conclusão, envio para a lixeira,

restauração, exclusão definitiva, persistência após sair e entrar, limpeza dos filtros ao sair e saída do usuário. Também foi testado o clique no cartão da tarefa, que expande a descrição, a prioridade, o prazo e o checklist de subtarefas na própria lista; uma subtarefa pode ser concluída nesse painel e o progresso é atualizado sem fechar o cartão. A Tabela 3 resume os principais testes.

Tabela 3 – Principais testes realizados.

Teste	**Resultado esperado**	**Resultado obtido**
Cadastrar usuário novo	201	201
Cadastrar usuário com e-mail repetido	409	409
Login com e-mail e senha corretos	200	200
Login com senha errada	401	401
Recuperar senha com e-mail e telefone corretos	200	200
Recuperar senha com telefone errado	401	401
Recuperar senha com menos de 6 caracteres	400	400
Entrar com a senha antiga após redefinir	401	401
Entrar com a nova senha	200	200
Criar tarefa com prazo, prioridade e categoria	201	201
Criar tarefa com prioridade inválida	400	400
Criar tarefa com categoria de outro usuário	400	400
Cadastrar categoria	201	201
Cadastrar categoria repetida	409	409
Criar subtarefa	201	201
Criar subtarefa sem título	400	400
Concluir subtarefa	200 e concluída = 1	200
Alterar subtarefa de tarefa na lixeira	400	400
Concluir tarefa	200 e status concluída	200
Mover tarefa para a lixeira	204	204
Restaurar tarefa	200	200
Excluir definitivamente tarefa ativa	400	400
Excluir definitivamente tarefa na lixeira	204	204
Consultar tarefa apagada	404	404
Buscar ignorando acentos e maiúsculas	Encontra a tarefa	Funcionou
Combinar busca, status e categoria	Somente tarefas que atendem a todos os filtros	Funcionou
Expandir tarefa na lista	Descrição, prioridade, prazo e subtarefas aparecem no cartão	Funcionou
Concluir subtarefa na lista	Progresso atualizado sem fechar o cartão	Funcionou
Criar categoria pelo modal de tarefa	Categoria criada e selecionada	Funcionou
Navegação com ícones SVG	Ícones alinhados e cor do item ativo	Funcionou
Navegar em tela pequena	Menu inferior acessa as telas	Funcionou
Fluxo completo na interface	Sem erros	Sem erros

Fonte: Elaborado pelos autores (2026).

Depois dos testes locais, o sistema foi publicado no Render Cloud e a URL pública foi verificada: a página e a API responderam 200.

4.2. Próximas etapas

O backend e o frontend foram publicados no Render Cloud e estão disponíveis em https://taskup-rffb.onrender.com/. Como o plano gratuito usa disco temporário, o banco de demonstração pode ser reiniciado a cada deploy. Como evolução, pretende-se emitir um token de sessão no login e proteger as rotas de tarefas e categorias, que hoje confiam no nome e no telefone enviados pela interface. Também é recomendável adicionar limite de tentativas na recuperação de senha e, futuramente, substituir a confirmação por e-mail e telefone por um link ou código temporário de uso único.

5. CONSIDERAÇÕES FINAIS

Nesta etapa, a API do TaskUp permite cadastrar e consultar usuários e categorias e executar operações CRUD em tarefas e subtarefas, com lixeira, restauração, organização por categorias e checklists. O cadastro e o login usam e-mail e senha, com a senha protegida por hash e salt, e a recuperação de senha confirma e-mail e telefone antes de gravar a nova senha. O banco valida os dados e vincula cada tarefa a um usuário. A interface web já consome a API e permite gerenciar categorias, buscar e filtrar tarefas e acompanhar o progresso das subtarefas. O sistema foi publicado no Render Cloud em https://taskup-rffb.onrender.com/. As próximas etapas são proteger as rotas com um token de sessão, adicionar limite de tentativas na recuperação de senha, implementar notificações de prazos e evoluir a persistência do banco de dados, hoje limitada ao disco temporário do plano gratuito.

6. REFERÊNCIAS

CORS. Cross-Origin Resource Sharing. MDN Web Docs. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/CORS. Acesso em: 8 nov. 2026.

EXPRESS. Express: Node.js web application framework. [S.l.: s.n., s.d.]. Disponível em: https://expressjs.com/. Acesso em: 8 nov. 2026.

FIELDING, Roy Thomas. Architectural styles and the design of network-based software architectures. 2000. Tese (Doutorado) – University of California, Irvine, 2000.

NODE.JS. Node.js documentation. [S.l.: s.n., s.d.]. Disponível em: https://nodejs.org/docs/latest-v22.x/api/. Acesso em: 8 nov. 2026.

OWASP. Password Storage Cheat Sheet. [S.l.: s.n., s.d.]. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Acesso em: 8 nov. 2026.

SQLITE. SQLite documentation. [S.l.: s.n., s.d.]. Disponível em: https://www.sqlite.org/docs.html. Acesso em: 8 nov. 2026.